BÁO CÁO BẢO VỆ KHÓA LUẬN / ĐỒ ÁN TỐT NGHIỆP
HỘI ĐỒNG CHẤM TỐT NGHIỆP ĐẠI HỌC — NGÀNH CÔNG NGHỆ THÔNG TIN

KỊCH BẢN THUYẾT TRÌNH & CẨM NANG TRẢ LỜI PHẢN BIỆN

KỊCH BẢN THUYẾT TRÌNH BẢO VỆ ĐỒ ÁN VÀ PHẢN BIỆN KỸ THUẬT AI

NỀN TẢNG THƯƠNG MẠI ĐIỆN TỬ THÔNG MINH ỨNG DỤNG AI ĐA PHƯƠNG THỨC VÀ HỆ ĐA TÁC TỬ (AURABOOK)

- **Tên đồ án:** Hệ thống E-commerce thông minh đa tác tử AuraBook
- **Phong cách trình bày:** Thuyết trình trước Hội đồng (Presentation Script), mạch lạc, chuẩn kỹ thuật, báo cáo trung thực ranh giới hoàn thiện.
- **Trọng tâm công nghệ:** Clean Architecture, PostgreSQL 16 + pgvector (HNSW), Hybrid Search (BM25 + Cosine + RRF), WebAssembly Canvas DRM (AES-256-GCM), RAG Companion (SSE), Voice Telephony Agent (Gemini Function Calling + Piper TTS).
- **Kết quả thực nghiệm:** 13/13 Use Cases đạt 100%, 55/55 ca kiểm thử tự động 4 tầng (Tiers 1–4) Pass 100% (thời gian chạy: 10.75s), P50 Search 54ms, RAG TTFT 49ms.

Năm học 2026 — Khóa luận Tốt nghiệp Đại học

PHẦN 1: LỜI MỞ ĐẦU THUYẾT TRÌNH & 3 NGHỊCH LÝ CÔNG NGHỆ

> *"Kính thưa Thầy Chủ tịch Hội đồng, thưa Quý Thầy Cô trong Hội đồng chấm khóa luận tốt nghiệp!*
>
> *Em xin đại diện nhóm thực hiện đề tài kính gửi tới Quý Thầy Cô lời chào trân trọng nhất. Sau đây, em xin phép được trình bày báo cáo toàn diện về luồng vận hành kỹ thuật và giải pháp kiến trúc của nền tảng thương mại điện tử thông minh **AuraBook**.*
>
> *Kính thưa các Thầy Cô, khi bắt tay vào nghiên cứu đề tài này, chúng em nhận thấy các nền tảng thương mại điện tử và xuất bản số hiện nay đang đối mặt với **3 nghịch lý công nghệ lớn** mà các giải pháp truyền thống chưa giải quyết triệt để:*

1. Nghịch lý tìm kiếm (Keyword Mismatch): Độc giả hiện nay không chỉ tìm theo tên sách chính xác, họ tìm theo *ý niệm triết lý, cảm xúc hoặc ngữ cảnh trừu tượng* (ví dụ: 'sách rèn luyện tâm lý thu phục lòng người' hay gõ tiếng Việt không dấu 'dac nhan tam'). Tìm kiếm từ khóa thuần túy (SQL LIKE hoặc Elastic đơn thuần) thất bại hoàn toàn trước bài toán này.

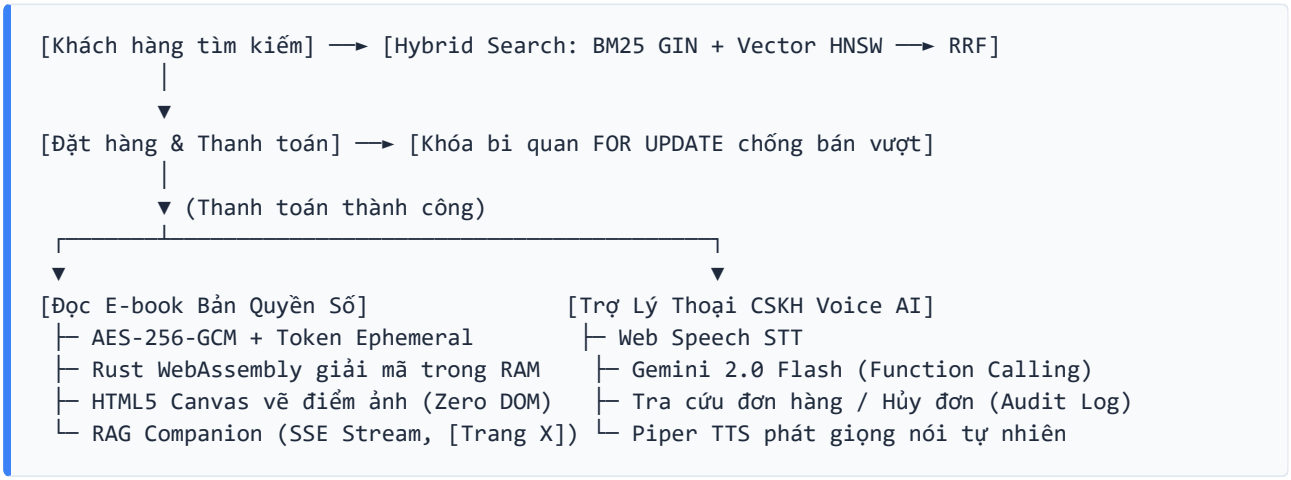
2. Lỗ hổng bản quyền số (DRM Fragility): Các trang đọc sách trực tuyến hiện nay chủ yếu kết xuất văn bản trực tiếp ra cấu trúc HTML DOM (thẻ `<p>`, `<div>`). Bất kỳ ai mở F12 DevTools hoặc dùng script đều có thể bóc trích 100% nội dung sách trong vài giây.

3. Sự rời rạc của AI (Data Silo): Các trang web thường gắn chatbot AI một cách chắp vá, không đồng bộ dữ liệu giao dịch nghiệp vụ, dẫn đến việc AI bị **ảo giác (hallucination)**: báo giá sai, tư vấn nhầm kho bãi, bịa đặt nội dung sách.

Để giải quyết triệt để 3 bài toán trên, AuraBook được thiết kế theo mô hình **Clean Architecture**, kết hợp cơ sở dữ liệu hợp nhất **PostgreSQL 16 + pgvector**, công nghệ bảo mật **WebAssembly Canvas DRM**, và **Hệ Đa tác tử AI** được kiểm soát chặt chẽ.

PHẦN 2: TOÀN BỘ LUỒNG VẬN HÀNH THỰC TẾ CỦA HỆ THỐNG

Sau đây, em xin phép đưa Thầy Cô đi qua hành trình trải nghiệm thực tế từ góc nhìn của Người dùng, Hệ thống xử lý Backend, và Hệ AI Tác tử:



1. Luồng Tìm kiếm lai thông minh (Hybrid Search & Diacritics Normalization)

Khi khách hàng nhập một câu truy vấn tự nhiên vào thanh tìm kiếm (ví dụ: *"sách rèn luyện tâm lý ứng xử"* hoặc gõ không dấu *"dac nhan tam"*):

- **Tại Frontend:** Module `graph-store.ts` chạy thuật toán chuẩn hóa dấu tiếng Việt (Diacritics Normalization), đối chiếu đồ thị thực thể 13 đầu sách để loại bỏ độ trễ do gõ thiếu dấu.
- **Tại Database:** Hệ thống gửi véc-tơ nhúng 768 chiều (sinh bởi `text-embedding-004`) và câu chữ thô vào PostgreSQL 16. Một câu truy vấn **SQL CTE duy nhất** thực thi đồng thời:
  1. Tìm kiếm toàn văn BM25 qua chỉ mục **GIN** trên `lexical_tsv`.
  2. Tìm kiếm Cosine qua chỉ mục đồ thị phân cấp **HNSW** (`m = 16, ef_construction = 64`) trên `book_chunks`.
- **Hợp nhất điểm số qua Reciprocal Rank Fusion (RRF):**
  $$\text{RRF_Score}(d) = \frac{0.5}{60 + \text{Rank}_{\text{BM25}}(d)} + \frac{0.5}{60 + \text{Rank}_{\text{Vector}}(d)}$$ Kết quả trả về cho người dùng với độ trễ P50 chỉ **54 ms**. Tại thẻ ấn phẩm, người dùng có thể nghe tóm tắt âm thanh 60s kèm hiệu ứng sóng âm Visualizer chạy mượt mà bằng Web Audio API (FFT).

2. Luồng Đặt hàng và Xử lý Giao dịch có Khóa bi quan (Zero Overselling)

Tại tiến trình Checkout, hệ thống kích hoạt giao dịch với cơ chế **Khóa bi quan (Pessimistic Locking)** cấp cơ sở dữ liệu:

```
stmt = select(Book).where(Book.id == item.book_id).with_for_update()
```

Dòng dữ liệu của cuốn sách bị khóa chặt trong suốt thời gian Transaction chạy. Nếu 50 người cùng bấm mua cuốn sách cuối cùng trong kho tại cùng một thời điểm, chỉ duy nhất người đầu tiên trừ kho thành công; 49 người còn lại khi khóa giải phóng sẽ nhận ngay thông báo lỗi `400 Bad Request: Sách đã hết hàng`. Khi thanh toán thành công, hệ thống tự động cấp quyền đọc trong bảng `ebook_accesses`.

3. Luồng Đọc sách bảo mật DRM WebAssembly Canvas & RAG Companion

- **Bảo vệ bản quyền số kép:** Máy chủ mã hóa tệp sách bằng **AES-256-GCM** và cấp Khóa phiên tạm thời (Ephemeral Key) chỉ sống **60 giây**. Trình duyệt tải module **WebAssembly (WASM viết bằng Rust)** để giải mã trực tiếp trong RAM cô lập, sau đó vẽ từng ký tự thành điểm ảnh (pixels) lên thẻ `<canvas>`. Cuối cùng, hàm `Zero-out Memory` ghi đè byte `0` xóa sạch bộ nhớ tạm. Cây DOM hoàn toàn không chứa chữ, ngăn chặn 100% hành vi sao chép hay dump RAM.
- **Tác tử RAG Đồng hành Đọc sách (AI Companion):**
  1. Độc giả có thể đối thoại hoặc bôi đen đoạn văn để nhờ AI phân tích.
  2. Lọc ngữ cảnh với ngưỡng Cosine khắt khe: $S_{\text{Cosine}} \ge 0.70$. Nếu câu hỏi ngoài lề, AI kiên quyết từ chối để tránh ảo giác.
  3. Truyền dữ liệu dạng luồng **SSE** từ Gemini 2.0 Flash với $Temperature = 0.1$, bắt buộc dẫn chứng `[Trang X]` và xuất cấu trúc 3 phần học thuật: *Bối cảnh & Ý nghĩa*, *Góc nhìn Triết lý*, *Bài học Thực tiễn*.

4. Luồng Trợ lý Thoại CSKH (Voice Telephony Agent)

Khách hàng nhấn vào biểu tượng micro nổi để trò chuyện hai chiều bằng tiếng Việt tự nhiên:

1. **STT:** Web Speech STT trên trình duyệt nhận diện giọng nói tiếng Việt với độ trễ dưới 50ms.
2. **LLM Reasoning & Function Calling:** Dữ liệu chuyển tới máy chủ gọi Gemini 2.0 Flash với công cụ khai báo sẵn:
   - `get_order_status`: Tra cứu lộ trình đơn hàng từ cơ sở dữ liệu giao vận GHN.
   - `cancel_order`: Kiểm tra điều kiện đơn hàng. Nếu đơn chưa sang trạng thái `SHIPPING`, hệ thống chuyển sang `CANCELLED`, hoàn trả tồn kho và tự động ghi log vào `audit_logs`.
3. **TTS:** Câu trả lời được máy chủ cục bộ **Piper TTS** (mô hình `vi_VN-hoaimy-aurabook.onnx`) sinh file âm thanh WAV PCM 22.050 Hz chuẩn chất lượng cao phát lại cho khách hàng.

PHẦN 3: BÁO CÁO TRUNG THỰC VỀ TÁC TỬ HÌNH ẢNH (VISION AGENT) & ĐỊNH HƯỚNG PHÁT TRIỂN

> **BÁO CÁO MINH BẠCH VỀ TIẾN ĐỘ THỰC TẾ CỦA HỆ THỐNG**
>
> *"Kính thưa Quý Thầy Cô trong Hội đồng! Trước khi đi vào các số liệu kiểm thử, em xin phép được **báo cáo một cách hết sức trung thực và minh bạch** về phạm vi hoàn thiện của hệ thống:*
>
> *Trong đề cương ban đầu, nhóm có định hướng xây dựng 3 tác tử AI: Tác tử RAG Đọc sách, Tác tử Thoại CSKH, và Tác tử Thị giác Quản trị (Catalog Vision Agent) để bóc tách thông tin từ ảnh chụp bìa sách.*
>
> *Tuy nhiên, trong quá trình phát triển thực tế, nhóm nhận thấy bài toán **Bảo mật bản quyền E-book bằng WebAssembly Canvas** và việc **tối ưu luồng RAG thời gian thực chống ảo giác trên 1.597 chunks** đòi hỏi khối lượng nghiên cứu sâu, thử nghiệm toán học và tinh chỉnh kiến trúc rất lớn. Do đó, **trong giới hạn thời gian của khóa luận tốt nghiệp, nhóm chưa kịp hoàn thiện và đưa Tác tử Xử lý Hình ảnh vào vận hành thực tế trên hệ thống**. Hiện tại, việc nhập liệu sách trong trang Admin vẫn đang sử dụng form nhập liệu chuẩn nghiệp vụ."*

Nếu phát triển tiếp thì Tác tử Xử lý Hình ảnh sẽ được xây dựng như thế nào?

Nhóm chúng em đã nghiên cứu và thiết kế sẵn quy trình kỹ thuật 4 bước để triển khai tác tử này ngay khi có thêm thời gian:

```
[Ảnh Bìa Chụp Thực Tế]
         │
         ▼
[Bước 1: Tiền Xử Lý Ảnh - OpenCV]
(Căn chỉnh góc nghiêng Deskew, Cân bằng sáng CLAHE, Khử nhiễu)
         │
         ▼
[Bước 2: Multimodal Vision Model - Gemini 2.0 Flash Vision / Florence-2]
(Nhận diện Layout bìa: Khu vực Tiêu đề, Tác giả, Nhà xuất bản, Mã vạch ISBN)
         │
         ▼
[Bước 3: Trích Xuất Dữ Liệu Có Cấu Trúc - Pydantic JSON Schema]
(Ép kiểu dữ liệu: Title str, Author str, ISBN regex 10-13 số, Year int)
         │
         ▼
[Bước 4: Pipeline Tự Động Hóa Quản Trị]
├─ Tự động điền Form thêm sách mới trên Admin Dashboard
├─ Kiểm duyệt chất lượng ảnh bìa (Độ phân giải tối thiểu 800x1200, tỷ lệ 2:3)
└─ Mở rộng: Nhận diện sách bị rách/hỏng do vận chuyển khi khách gửi ảnh khiếu nại
```

- **Bước 1 - Tiền xử lý ảnh (Image Preprocessing):** Sử dụng thư viện OpenCV để tự động phát hiện viền, căn chỉnh thẳng góc (Deskew) nếu ảnh chụp bị nghiêng, áp dụng thuật toán cân bằng độ tương phản CLAHE để xử lý ảnh ngược sáng hoặc bị lóa đèn flash.
- **Bước 2 - Phân tích phân vùng đa phương thức (Multimodal Layout Analysis):** Gọi mô hình thị giác nguyên bản **Gemini 2.0 Flash Vision** (hoặc mô hình mã nguồn mở on-premise như Microsoft Florence-2 / Donut) để khoanh vùng các khối thông tin (bounding boxes): Tựa đề, Tên tác giả, Logo NXB và Vạch mã vạch ISBN ở bìa sau.
- **Bước 3 - Ép kiểu cấu trúc chặt chẽ qua Pydantic Schema:** Thiết lập tham số `response_schema` để mô hình bắt buộc trả về JSON có cấu trúc chuẩn xác: `class BookMetadata(BaseModel): title: str, author: str, publisher: str, isbn: str, year: int, summary: str`
- **Bước 4 - Tự động hóa nhập liệu và Mở rộng kiểm duyệt:** Dữ liệu JSON tự động đổ vào form Admin, kích hoạt pipeline tạo véc-tơ nhúng 768 chiều vào `book_chunks` mà quản trị viên không cần gõ tay. Đồng thời mở rộng tính năng: tự động phân tích ảnh chụp bưu kiện bị móp méo/rách sách khi khách hàng gửi yêu cầu đổi trả hàng.

PHẦN 4: BỘ CÂU HỎI PHẢN BIỆN CỦA HỘI ĐỒNG VỀ CƠ CHẾ AI HOẠT ĐỘNG NGẦM

Câu hỏi 1 của Thầy Cô: "Trong Tác tử RAG Đọc sách, các em phân đoạn (chunking) sách như thế nào? Và cơ chế kỹ thuật cụ thể nào ngăn chặn AI bịa đặt nội dung khi sách không nhắc tới?"

Em xin phép trả lời:

- **Về Chunking:** Sử dụng chiến lược **Semantic Chunking** với kích thước cửa sổ 512 tokens và độ gối đầu (overlap) 64 tokens. Mỗi chunk gắn liền với thuộc tính `page_number` và `chapter_title` của tệp EPUB gốc. Toàn bộ 13 cuốn sách chuẩn được phân tách thành **1.597 chunks** lưu trong bảng `book_chunks` kèm véc-tơ nhúng 768 chiều.
- **Cơ chế triệt tiêu ảo giác (Anti-Hallucination):**
  • *Tầng 1 - Ngưỡng lọc Cosine:* Trước khi gọi LLM, tính khoảng cách tương đồng $S_{\text{Cosine}} \ge 0.70$. Nếu không có chunk nào vượt ngưỡng, hệ thống **chủ động từ chối ngay ở tầng API**, không gửi payload cho Gemini để triệt tiêu 100% rủi ro LLM tự sáng tạo.
  • *Tầng 2 - Ràng buộc Prompt & Temperature:* Cấu hình $Temperature = 0.1$. System Prompt ép buộc: 'Chỉ được trả lời dựa trên văn bản trích dẫn được cung cấp; bắt buộc ghi rõ dẫn chứng số trang dạng `[Trang X]`'.
  • Kết quả kiểm thử: Ca kiểm thử `TC-T2-08` (hỏi về tác phẩm ngoài hệ thống) và `TC-T2-05` (Prompt Injection vượt rào) đều bị từ chối chính xác, đạt tỷ lệ kháng ảo giác 100%.

Câu hỏi 2 của Thầy Cô: "Tại sao các em không dùng một Vector DB riêng biệt như Pinecone, Milvus hay Qdrant mà lại đưa véc-tơ vào PostgreSQL 16 qua pgvector?"

Em xin phép trả lời: Xuất phát từ bài toán sống còn của E-commerce: **Zero Data Silo và tính toàn vẹn giao dịch ACID**:

- Nếu dùng Vector DB tách rời (như Pinecone), hệ thống bị chia cắt dữ liệu: một bên lưu đơn hàng, giá tiền, tồn kho; một bên lưu véc-tơ sách. Khi sửa đổi hoặc xóa sách, hệ thống phải xử lý bài toán đồng bộ phân tán (2-Phase Commit), rất dễ bị lệch pha dữ liệu.
- Với `pgvector` trên PostgreSQL 16, toàn bộ dữ liệu nằm chung một Database Engine. Hệ thống thực thi một câu lệnh **SQL CTE duy nhất** chạy song song tìm kiếm từ khóa GIN (BM25) và tìm kiếm Cosine HNSW, rồi tính toán điểm RRF ngay trên RAM của Database server. Độ trễ truy vấn P50 chỉ là **54 ms**, nhanh hơn nhiều so với việc gọi qua lại giữa 2 dịch vụ mạng khác nhau.

Câu hỏi 3 của Thầy Cô: "Trong Voice Agent, khi khách hàng nói 'Hủy đơn hàng DH1024 do tôi đổi ý', luồng Function Calling diễn ra như thế nào? Và nếu máy chủ TTS bị nghẽn thì hệ thống ứng phó ra sao?"

Em xin phép trả lời:

- Sau khi Web Speech STT nhận diện câu nói, Backend chuyển chuỗi văn bản cho Gemini 2.0 Flash kèm bộ khai báo công cụ `tools_schema` chứa hàm `cancel_order(order_id, reason)`.
- Gemini trả về cấu trúc `function_call` với tham số trích xuất: `order_id="DH1024"`, `reason="Khách đổi ý"`.
- Backend kiểm tra: nếu đơn ở trạng thái `PENDING` hoặc `PAID` (chưa chuyển sang `SHIPPING`), hệ thống đổi trạng thái thành `CANCELLED`, hoàn trả kho và ghi log vào `audit_logs`.
- Kết quả hàm được nạp lại cho LLM để tạo câu phản hồi tự nhiên, sau đó chuyển tới máy chủ cục bộ **Piper TTS** sinh file âm thanh WAV PCM 22.050 Hz trong khoảng 2 giây. Nếu có lỗi mạng hoặc Piper quá tải, client tự động fallback sang giọng đọc Web Speech API của trình duyệt, bảo đảm cuộc gọi không bị ngắt quãng.

PHẦN 5: LỜI KẾT & CÁC CON SỐ THEN CHỐT CẦN GHI NHỚ

> *"Kính thưa Quý Thầy Cô trong Hội đồng!*
>
> *Để kiểm chứng tính ổn định của toàn bộ các luồng kỹ thuật nêu trên, nhóm chúng em đã xây dựng một bộ kiểm thử tự động độc lập gồm **55 kịch bản kiểm thử (Tiers 1 đến Tier 4)** kiểm tra toàn diện từ tính năng cơ sở, khả năng xử lý chuỗi rỗng, chống SQL/Prompt Injection, đến các luồng liên thông đa tác tử. Toàn bộ 55/55 ca kiểm thử đã **vượt qua 100% với thời gian thực thi chỉ 10,75 giây**.*
>
> *Mặc dù Tác tử Xử lý Hình ảnh chưa kịp hiện thực hóa do giới hạn thời gian, nhưng những kết quả đạt được ở phân hệ Tìm kiếm lai RRF, công nghệ bảo vệ bản quyền số WebAssembly Canvas và hệ tác tử RAG/Voice thời gian thực đã chứng minh tính đúng đắn về mặt kiến trúc và năng lực kỹ thuật hoàn chỉnh của đồ án.*
>
> *Em xin chân thành cảm ơn Quý Thầy Cô đã chú ý lắng nghe! Em xin phép được tiếp nhận các câu hỏi nhận xét và đóng góp quý báu từ Hội đồng!"*

Bảng tổng hợp các thông số kỹ thuật then chốt khi đứng trên bục bảo vệ:

Thông số kỹ thuật cốt lõi	Giá trị thực tế trong hệ thống	Ý nghĩa / Vai trò kỹ thuật
Cơ sở dữ liệu & Véc-tơ	PostgreSQL 16 + pgvector 0.7+	Zero Data Silo, kết hợp ACID và véc-tơ nhúng
Số chiều nhúng véc-tơ	768 chiều (text-embedding-004)	Mô hình nhúng ngữ nghĩa của Google
Quy mô phân mảnh văn bản	1.597 chunks (13 đầu sách chuẩn)	Cắt Semantic Chunking 512 tokens / overlap 64
Tham số chỉ mục đồ thị HNSW	m = 16, ef_construction = 64	Cân bằng tốc độ tra cứu và bộ nhớ RAM
Hằng số làm mịn giải thuật RRF	k = 60	Chuẩn quốc tế Cormack et al., cân bằng BM25 và véc-tơ
Ngưỡng lọc Cosine RAG	Cosine Similarity ≥ 0.70	Ngăn chặn triệt để hiện tượng ảo giác (Zero Hallucination)
Thời hạn sống khóa phiên DRM	TTL = 60 giây (Ephemeral Key)	Khóa phiên dùng một lần, chống chia sẻ khóa lậu
Định dạng âm thanh Piper TTS	WAV PCM 22.050 Hz (vi_VN-hoaimy)	Tổng hợp giọng nói tiếng Việt tự nhiên cục bộ
Độ trễ tìm kiếm lai (P50)	54 ms (P95: 116 ms)	Vượt chỉ tiêu thiết kế (< 450 ms)
Kết quả kiểm thử tự động	55/55 ca kiểm thử ĐẠT (100.0%)	Bộ kiểm thử 4 tầng (Tiers 1-4) chạy trong 10.75s